Prompt: Build MyAssistant — OpenClaw-style AI Assistant in Python + Google ADK

> Paste this entire prompt into Claude Code, Cursor, Windsurf, or any AI editor. The AI will scaffold the complete application end-to-end.

YOUR TASK

Build a complete, production-ready, locally-run **personal AI assistant** called `myassistant` in Python.

It is modelled after the architecture of OpenClaw (https://github.com/openclaw/openclaw) but implemented fully in Python using **Google ADK** (Agent Development Kit) for agent orchestration. The app runs as a **single async Python process** (FastAPI + asyncio), connects to **Telegram** as the first messaging channel, and exposes a **CLI mode** for local terminal testing.

STRICT REQUIREMENTS

- Language: **Python 3.11+** only
- Agent framework: **Google ADK** (`google-adk` package)
- Web/async framework: **FastAPI** + **uvicorn** + **asyncio**
- Messaging: **python-telegram-bot v21+** (async)
- Database: **SQLite** via Python's built-in `sqlite3` (no ORM)
- Browser automation: **Playwright** (async)
- Config: **python-dotenv** + `.env` file
- Packaging: `pyproject.toml` (no `setup.py`)
- **Every function and class must have a docstring** explaining what it does
- **Every module must have a top-level docstring** explaining its role in the system
- Inline comments on any logic that is non-obvious
- Create a `README.md` with full setup and run instructions
- Create a `DESIGN.md` with architecture explanation and data flow diagrams (ASCII)

ARCHITECTURE OVERVIEW

```
One Python Process (asyncio)
│
├── gateway/
│   ├── server.py          ← FastAPI app + WebSocket endpoint (:8765)
│   ├── router.py          ← routes normalised messages to ADK agent
│   └── session_manager.py ← SQLite-backed session CRUD
│
├── channels/
│   ├── base.py            ← abstract BaseAdapter
│   ├── telegram/
│   │   └── adapter.py     ← python-telegram-bot handler
```

```
│   └── cli/
│       └── adapter.py     ← stdin/stdout adapter for local testing
│
├── agent/
│   ├── agent.py           ← Google ADK Agent definition + tool registration
│   ├── runner.py          ← Google ADK Runner wrapper (async streaming)
│   └── tools/
│       ├── web_search.py  ← DuckDuckGo search via httpx
│       ├── browser.py     ← Playwright page fetch + screenshot
│       └── memory.py      ← SQLite-backed remember/recall
│
├── skills/
│   ├── weather.py         ← wttr.in weather skill
│   └── calculator.py      ← safe expression evaluator skill
│
├── storage/
│   ├── db.py              ← SQLite connection, migrations, helper queries
│   └── models.py          ← Session, Message, Memory dataclasses
│
├── config/
│   └── settings.py        ← loads .env, exposes typed Settings object
│
├── main.py                ← entry point — wires everything, starts event loop
├── .env.example           ← template for environment variables
├── pyproject.toml         ← dependencies and project metadata
├── README.md              ← setup, install, run instructions
└── DESIGN.md              ← architecture, data flow, component descriptions
```

DETAILED COMPONENT SPECIFICATIONS

1. `storage/models.py`

Define these dataclasses:

```python
@dataclass
class Session:
    id: str                  # UUID4
    user_id: str             # platform user identifier
    channel: str             # "telegram" | "cli" | "whatsapp"
    model: str               # default: "gemini-2.0-flash"
    created_at: datetime
    last_active: datetime

@dataclass
class Message:
    id: str
    session_id: str
    role: str                # "user" | "assistant" | "tool"
    content: str
    timestamp: datetime
```

```python
@dataclass
class Memory:
    id: str
    user_id: str
    key: str                # e.g. "user_name", "preferred_language"
    value: str
    updated_at: datetime
```

---

2. `storage/db.py`

- Class `Database` with `__init__(db_path: str)`
- Method `migrate()` — creates all tables on first run using `CREATE TABLE IF NOT EXISTS`
- Method `get_or_create_session(user_id, channel) -> Session`
- Method `save_message(session_id, role, content) -> Message`
- Method `get_recent_messages(session_id, limit=20) -> list[Message]`
- Method `set_memory(user_id, key, value)`
- Method `get_memory(user_id, key) -> str | None`
- Method `get_all_memories(user_id) -> dict[str, str]`
- Use `threading.Lock` for thread safety
- All queries use parameterised statements (no f-string SQL)

---

3. `config/settings.py`

- Load `.env` using `python-dotenv`
- Expose a `Settings` dataclass with:
  - `TELEGRAM_TOKEN: str`
  - `GOOGLE_API_KEY: str` (for ADK / Gemini)
  - `GATEWAY_PORT: int = 8765`
  - `DB_PATH: str = "~/.myassistant/myassistant.db"`
  - `DEFAULT_MODEL: str = "gemini-2.0-flash"`
  - `LOG_LEVEL: str = "INFO"`
- Raise `ValueError` with a helpful message if required keys are missing

---

4. `agent/tools/web_search.py`

```python
async def web_search(query: str) -> str:
    """
    Search the web using DuckDuckGo and return a plain-text summary of the top results.
    Uses the DuckDuckGo Instant Answer API (no API key required).
    Returns up to 3 result summaries joined as a string.
    """
```

Use `httpx.AsyncClient` to call `https://api.duckduckgo.com/?q={query}&format=json&no_html=1`

---

5. `agent/tools/browser.py`

```
async def fetch_page(url: str) -> str:
    """
    Open a URL in a headless Playwright browser and return the page's
    visible text content (stripped of HTML). Useful for reading articles,
    docs, or any web page the user links to.
    """

async def take_screenshot(url: str) -> str:
    """
    Navigate to a URL and take a full-page screenshot.
    Saves it to /tmp/screenshot.png and returns the file path.
    """
```

Use `async_playwright` context manager. Launch Chromium in headless mode.

---

6. `agent/tools/memory.py`

```
async def remember(key: str, value: str, user_id: str) -> str:
    """
    Store a piece of information about the user permanently in SQLite.
    Example: remember("name", "Amit", user_id) stores the user's name.
    Returns a confirmation string.
    """

async def recall(key: str, user_id: str) -> str:
    """
    Retrieve a previously stored piece of information about the user.
    Returns the stored value, or a message saying it was not found.
    """

async def recall_all(user_id: str) -> str:
    """
    Return all stored memories for this user as a formatted string.
    Useful for the agent to load context at the start of a conversation.
    """
```

These tools need access to the `Database` instance — inject it via a closure or a module-level singleton. Document the chosen approach clearly.

---

7. `skills/weather.py`

```
async def get_weather(location: str) -> str:
    """
    Fetch current weather for a location using the free wttr.in API.
    No API key required. Returns a concise one-line weather summary.
    Example: get_weather("London") -> "London: 🌥 14°C, Partly cloudy, Wind
18km/h"
    """
```

Use `httpx.AsyncClient` to call `https://wttr.in/{location}?format=3`

---

8. `skills/calculator.py`

```
def calculate(expression: str) -> str:
    """
    Safely evaluate a mathematical expression and return the result as a string.
    Uses Python's ast module to parse and evaluate — never uses eval() directly.
    Only supports: +, -, *, /, **, (), numbers, and basic math functions.
    Example: calculate("2 ** 10 + 5 * 3") -> "1039"
    """
```

Implement a whitelist-based AST node visitor to prevent code injection.

---

9. `agent/agent.py`

```
from google.adk.agents import Agent
from google.adk.tools import FunctionTool

def build_agent(db) -> Agent:
    """
    Construct and return the Google ADK Agent with all tools registered.

    The agent is given a system instruction that tells it:
    - It is a local personal assistant
    - It should be concise (replies go to messaging apps)
    - It should use memory tools to remember user preferences
    - It should not use markdown headers or bullet lists unless asked

    Tools registered:
    - web_search: search the internet
    - fetch_page: read any web URL
    - remember / recall / recall_all: persistent user memory
    - get_weather: current weather
    - calculate: safe math evaluation
    """
```

The system instruction must explicitly tell the model that its output will be sent to a messaging app, so responses should be conversational and concise (no markdown unless asked).

---

10. `agent/runner.py`

```
class AgentRunner:
    """
    Wraps the Google ADK Runner to provide a simple async interface.
    Handles session creation with ADK's InMemorySessionService,
    runs the agent, and returns the final text response.
    """

    def __init__(self, agent: Agent):
        """Initialise the ADK Runner and session service."""

    async def run(self, session_id: str, user_id: str, message: str) -> str:
        """
        Run the agent for a given session and user message.
        Creates an ADK session if one doesn't exist yet.
        Returns the agent's final text response as a string.
        Logs token usage if available.
        """
```

Use `runner.run_async()` and iterate events to find `event.is_final_response()`.

---

11. `gateway/session_manager.py`

```
class SessionManager:
    """
    Manages conversation sessions stored in SQLite.
    A session ties together a user, their channel (Telegram/CLI),
    their preferred model, and their conversation history.
    """

    def get_or_create(self, user_id: str, channel: str) -> Session:
        """Return existing session for this user+channel, or create a new one."""

    def touch(self, session_id: str):
        """Update last_active timestamp to now."""

    def set_model(self, session_id: str, model: str):
        """Change the LLM model used for this session."""

    def get_session(self, session_id: str) -> Session | None:
        """Retrieve session by ID. Returns None if not found."""
```

---

12. `gateway/router.py`

```
class MessageRouter:
    """
    Central routing logic. Receives a normalised incoming message from any
    channel adapter, runs it through the ADK agent, and returns the reply.

    This is the core of the hub-and-spoke architecture:
    every channel adapter calls router.handle() and gets back a string reply.
    """

    def __init__(self, agent_runner: AgentRunner, session_manager:
SessionManager):
        ...

    async def handle(self, user_id: str, channel: str, text: str) -> str:
        """
        1. Get or create a session for this user+channel
        2. Log the incoming message to the DB
        3. Run the ADK agent
        4. Log the reply to the DB
        5. Touch the session (update last_active)
        6. Return the reply string
        """

    async def handle_command(self, user_id: str, text: str) -> str | None:
        """
        Check if the message is a slash command and handle it.
        Supported commands:
          /start or /help  -> welcome message + list of commands
          /model <name>    -> switch the LLM model for this session
          /memory          -> show all stored memories for this user
          /clear           -> clear conversation history (start fresh)
        Returns the command response string, or None if not a command.
        """
```

---

13. `channels/base.py`

```
class BaseAdapter(ABC):
    """
    Abstract base class for all channel adapters.
    Each adapter translates between a specific messaging platform's
    protocol and the gateway's normalised message format.
    """

    @abstractmethod
    async def start(self, router: MessageRouter) -> None:
        """Start listening for messages. Must run indefinitely."""
```

```python
    @abstractmethod
    async def send_message(self, user_id: str, text: str) -> None:
        """Send a text reply to the user on this platform."""

    def split_long_message(self, text: str, max_length: int = 4096) -> list[str]:
        """
        Split a long response into chunks that fit within platform limits.
        Splits on sentence boundaries where possible.
        Default max_length is Telegram's limit (4096 chars).
        """
```

---

14. channels/cli/adapter.py

```python
class CLIAdapter(BaseAdapter):
    """
    A stdin/stdout adapter for testing the assistant from the terminal.
    Useful for development — no API tokens needed.
    User ID is fixed as "cli_user". Channel is "cli".
    Supports the same slash commands as any other channel.
    Shows a prompt "You: " and prints "Assistant: <reply>".
    """

    async def start(self, router: MessageRouter) -> None:
        """Read from stdin in a loop, send to router, print reply."""
```

---

15. channels/telegram/adapter.py

```python
class TelegramAdapter(BaseAdapter):
    """
    Telegram channel adapter using python-telegram-bot v21 (async).
    Uses long-polling (no webhook needed for local dev).

    Handles:
    - Text messages -> router.handle()
    - /start, /help, /model, /memory, /clear commands -> router.handle_command()
    - Sends "typing..." action while the agent is thinking
    - Splits long responses into multiple messages if needed
    """

    def __init__(self, token: str):
        """Initialise the Telegram Application with the bot token."""

    async def start(self, router: MessageRouter) -> None:
        """Register handlers and start polling. Runs until cancelled."""
```

---

16. `gateway/server.py`

```
"""
Optional FastAPI WebSocket gateway server.
External tools (a future web UI, CLI scripts, mobile apps) can connect
to ws://localhost:8765 and send/receive JSON messages.

Message format (inbound):
{
    "user_id": "string",
    "channel": "websocket",
    "text": "string"
}

Message format (outbound):
{
    "session_id": "string",
    "text": "string",
    "timestamp": "ISO8601"
}
"""
```

- FastAPI app with a `/ws` WebSocket endpoint
- `/health` HTTP endpoint returning `{"status": "ok"}`
- `/sessions` HTTP endpoint listing active sessions
- Run via uvicorn in the background as an asyncio task

---

17. `main.py`

```
"""
Entry point for myassistant.
Wires together all components and starts the asyncio event loop.

Startup sequence:
1. Load settings from .env
2. Initialise and migrate SQLite database
3. Build the Google ADK agent with all tools
4. Create AgentRunner and SessionManager
5. Create MessageRouter
6. Start selected channel adapters concurrently with asyncio.gather()
7. Optionally start the FastAPI WebSocket server

Usage:
    python main.py --channel cli        # terminal mode
    python main.py --channel telegram   # telegram bot mode
    python main.py --channel all        # all channels + WS server
"""
```

Use `argparse` for the `--channel` flag.

---

18. `pyproject.toml`

```toml
[project]
name = "myassistant"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "google-adk>=0.1.0",
    "fastapi>=0.111.0",
    "uvicorn>=0.29.0",
    "python-telegram-bot>=21.0",
    "httpx>=0.27.0",
    "playwright>=1.44.0",
    "python-dotenv>=1.0.0",
]

[project.optional-dependencies]
dev = ["pytest>=8.0", "pytest-asyncio>=0.23", "black", "ruff"]

[project.scripts]
myassistant = "main:cli_entry"
```

---

19. `.env.example`

```
# Required
TELEGRAM_TOKEN=your_telegram_bot_token_here
GOOGLE_API_KEY=your_google_api_key_here

# Optional
DEFAULT_MODEL=gemini-2.0-flash
GATEWAY_PORT=8765
DB_PATH=~/.myassistant/myassistant.db
LOG_LEVEL=INFO
```

---

20. `README.md` — must include all of the following sections:

    1. **What is myassistant?** — one paragraph overview
    2. **Architecture** — ASCII diagram of the component structure
    3. **Prerequisites** — Python 3.11+, a Telegram bot token, a Google API key
    4. **Installation**

```
git clone ...
cd myassistant
python -m venv .venv
source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -e .
playwright install chromium
cp .env.example .env
# fill in .env with your keys
```

5. **Getting a Telegram Bot Token** — step-by-step via BotFather
6. **Getting a Google API Key** — link to Google AI Studio
7. **Running the app**

```
python main.py --channel cli        # test locally first
python main.py --channel telegram   # full bot
python main.py --channel all        # everything
```

8. **Available slash commands** — table of /help, /model, /memory, /clear
9. **Adding a new skill** — walkthrough showing how to add a new FunctionTool
10. **Running as a background service** — systemd unit file example (Linux) and launchd plist example (macOS)
11. **Troubleshooting** — common errors and fixes

---

21. `DESIGN.md` — must include all of the following sections:

1. **Design Philosophy** — why single-process, why Google ADK, why SQLite
2. **Component Map** — full ASCII directory tree with one-line description per file
3. **Process Model** — explain this is ONE Python process using asyncio.gather(), not microservices
4. **Data Flow Diagram** — ASCII art showing the 9-step message journey from user → platform → adapter → router → ADK agent → LLM → reply
5. **Session Lifecycle** — how a session is created, used, and persisted
6. **Tool vs Skill** — explain the difference: tools are low-level (search, browser); skills are higher-level domain capabilities built on top of tools
7. **How Google ADK Fits In** — which ADK classes are used (Agent, Runner, InMemorySessionService, FunctionTool) and why
8. **Adding a New Channel** — step-by-step guide: create adapter subclass, implement start() and send_message(), register in main.py
9. **Security Considerations** — local-only by default, how to expose externally safely
10. **Future Enhancements** — WebSocket gateway for external apps, vector memory with chromadb, multi-agent sub-tasks with ADK ParallelAgent

---

CODE QUALITY RULES

Apply these rules to every file generated:

- Every **module** starts with a docstring explaining its role
- Every **class** has a docstring explaining its responsibility
- Every **method/function** has a docstring with: what it does, key parameters, what it returns
- Inline comments on any `if/else` logic that is not self-explanatory
- No magic numbers — use named constants or settings
- No hardcoded strings for error messages — use constants at top of file
- Use `logging` module (not `print`) everywhere except CLI adapter output
- All async functions must be properly awaited — no fire-and-forget unless documented
- Handle exceptions with specific `except` clauses, never bare `except:`
- Type hints on every function signature

EXAMPLE: How a Telegram message flows through the system

Use this as a reference when deciding how components connect:

```
[User types "what's the weather in London?" in Telegram]
         |
         | (Telegram delivers update via long-poll)
         v
TelegramAdapter.on_message()
  - extracts user_id = "123456789"
  - extracts text = "what's the weather in London?"
  - sends "typing..." action to Telegram
  - calls await router.handle("123456789", "telegram", "what's the weather...")
         |
         v
MessageRouter.handle()
  - calls session_manager.get_or_create("123456789", "telegram")
    -> returns or creates Session(id="sess_abc", user_id="123456789", ...)
  - logs incoming message to DB
  - calls await agent_runner.run("sess_abc", "123456789", "what's the weather...")
         |
         v
AgentRunner.run()
  - creates or reuses ADK session for "sess_abc"
  - wraps message in ADK Content/Part format
  - calls runner.run_async() — streams events
  - ADK agent decides to call get_weather("London")
         |
         v
get_weather("London")  [tool execution]
  - calls https://wttr.in/London?format=3
  - returns "London: 🌥 14°C, Partly cloudy, Wind 18km/h"
         |
         v
ADK agent receives tool result, generates final response:
  "It's currently 14°C and partly cloudy in London, with winds at 18 km/h."
         |
         v
AgentRunner returns reply string to MessageRouter
```

```
          |
          v
MessageRouter
  - logs reply to DB
  - touches session
  - returns reply to TelegramAdapter
          |
          v
TelegramAdapter.send_message("123456789", "It's currently 14°C...")
  - splits if > 4096 chars
  - sends via telegram bot API
          |
          v
[User sees the reply in Telegram]
```

WHAT TO BUILD FIRST (in order)

The AI editor should build components in this exact order so each step is testable:

1. `pyproject.toml` + `.env.example`
2. `config/settings.py`
3. `storage/models.py` + `storage/db.py` (with migration)
4. `agent/tools/memory.py` (depends on db)
5. `agent/tools/web_search.py` (no deps)
6. `skills/weather.py` (no deps)
7. `skills/calculator.py` (no deps)
8. `agent/tools/browser.py` (needs Playwright)
9. `agent/agent.py` (assembles all tools)
10. `agent/runner.py` (wraps ADK Runner)
11. `gateway/session_manager.py`
12. `gateway/router.py`
13. `channels/base.py`
14. `channels/cli/adapter.py` ← test everything up to here with CLI before touching Telegram
15. `channels/telegram/adapter.py`
16. `gateway/server.py`
17. `main.py`
18. `README.md`
19. `DESIGN.md`

ACCEPTANCE CRITERIA

The implementation is complete when:

- ☐ `python main.py --channel cli` starts a working terminal chatbot
- ☐ The bot answers "what is 2+2" → "4" using the calculator tool
- ☐ The bot answers "weather in London" → real weather via wttr.in
- ☐ The bot answers "search for Python asyncio tutorial" → summarised results

- [ ] `/memory` command shows stored key-value facts
- [ ] `/model gemini-1.5-pro` switches the model for the session
- [ ] `python main.py --channel telegram` works with a real Telegram bot
- [ ] All functions have docstrings
- [ ] `README.md` exists and covers all 11 sections listed above
- [ ] `DESIGN.md` exists and covers all 10 sections listed above
- [ ] No hardcoded API keys anywhere in code